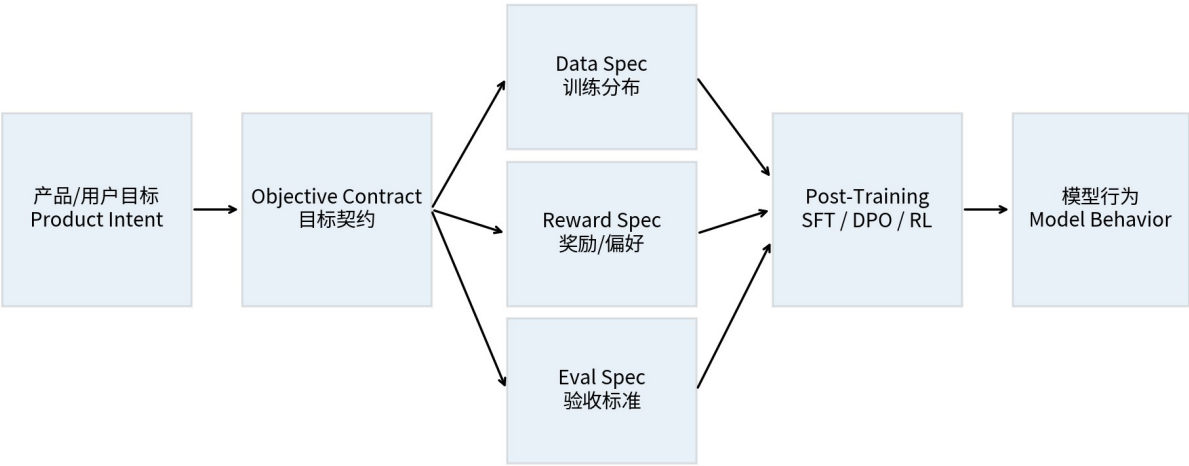


LLM 后训练之目标定义

从产品意图到 Capability / Behavior / Reward / Eval Contract

Deep Research & Engineering Handbook

Objective Compiler: 一份目标规范，编译成 Data / Reward / Eval 三种工程产物



研究范围：2021-2026 公开论文、官方技术文档与工程实践

研究日期：2026-08-13

核心结论 目标定义不是一句“提高模型能力”的 KPI，而是一份可执行的 Objective Contract：它必须能被编译为训练数据分布、奖励/偏好信号、评测标准、上线门槛和失败归因体系。

执行摘要

LLM 后训练的第一性问题不是“用 SFT、DPO 还是 GRPO”，而是“我们究竟要把模型训练成什么”。只要目标定义含糊，算法越强、算力越多，越可能把模型快速优化到错误方向。InstructGPT 明确指出，预训练的 next-token objective 与“helpfully and safely follow user instructions”并不相同；后训练因此首先是在重新定义行为目标。[1]

目标定义最容易被误解成一个高层愿景，例如“更智能”“更安全”“更会写代码”。真正可训练的目标必须继续下钻：谁是用用户、什么任务分布、什么行为算成功、什么行为绝不能出现、不同目标冲突时谁优先、如何被 grader/verifier 测量、如何进入 reward，以及什么指标达到后才允许上线。

公开研究显示，多目标之间经常存在真实张力。Anthropic 在 helpfulness/harmlessness RLHF 中观察到过度优化 harmlessness 会得到频繁、夸张的保守回答，并明确展示了两者在数据混合与损失权重上的冲突。[2] OpenAI 后续的 Rule-Based Rewards、Deliberative Alignment 和 safe-completion 则体现了一个更成熟的方向：把行为规范写成可解释规则或 specification，再让规则参与数据生成、奖励和评测。[5][6][7]

本手册采用的核心模型 Objective = Mission / Product Objective + Capability Spec + Behavior Spec + Constraint & Risk Spec + Task Distribution + Reward/Eval Contract。上层负责“为什么”，下层负责“如何可训练、可测量、可迭代”。

问题	错误做法	正确工程问题
“提高 Coding 能力”	只提高 HumanEval/单测分数	需要提高哪类代码任务？单文件生成、repo 理解、bug 定位、patch、test、debug、tool use 还是 long-horizon completion？
“更安全”	拒绝率越高越好	哪些输入/输出构成约束？什么情况下应完成、限制性完成、澄清或拒绝？安全与效用如何共同验收？
“更会推理”	让回答更长	目标是最终正确率、样本效率、验证能力、反思、工具使用，还是固定 token budget 下的成功率？
“更像用户想要的”	只采集偏好 pair	用户是谁？产品开发者与最终用户冲突时谁优先？偏好是否代表 truthfulness / task success？

目录与阅读地图

章节	核心问题	输出物
1. 为什么目标定义是最上游模块	为什么不能先选算法？	Objective Contract
2. 目标定义六层模型	高层愿景如何逐层可执行？	Mission/Capability/Behavior/ Constraint/Task/Eval
3. Capability Spec	“会什么” 如何分解？	Capability Matrix
4. Behavior Spec	“怎么做” 如何规范？	Behavior Rules / Defaults
5. 多目标与冲突	helpful/safe/fast/correct 如何权衡？	Priority + Pareto + Constraints
6. Objective -> Data	目标如何决定训练分布？	Task Mixture / Sampling Policy
7. Objective -> Reward	目标如何变成训练信号？	Reward Contract
8. Objective -> Eval	如何避免训练指标与产品目标错位？	Eval Spec / Gate
9. Coding Agent 案例	“提高 Coding” 如何完整拆解？	Coding Objective Tree
10. Reasoning / Chat 案例	不同模型类型如何不同定义？	Domain-specific Objective
11. 失败模式	目标定义常见哪些坑？	Failure Taxonomy
12. 工程模板	如何真正落地？	YAML / Review / Checklist
13. 成熟度模型	团队如何从 KPI 升级为 Objective System？	Maturity Model

1. 为什么“目标定义”是后训练最上游模块

预训练优化的是语言建模目标，而产品真正需要的是任务行为目标。InstructGPT 的关键论点是：模型越大并不自动意味着越能遵循用户意图，原因在于 next-token prediction 与用户希望模型“有帮助、诚实、无害”的目标并不等价。[1] 这意味着后训练不是简单增加数据，而是在改变优化问题本身。

因此，Objective 不是训练流水线之外的产品文档，而是训练系统的上游配置。它决定：哪些 prompt 值得收集；什么 completion 是正例；chosen/rejected 如何定义；reward 的各分量如何构造；哪些 benchmark 属于主指标；哪些行为属于 hard constraint；以及出现 trade-off 时谁让步。



越往下越可测量、可生成数据、可计算奖励；越往上越接近产品价值与治理目标

资料依据：InstructGPT [1]；OpenAI Model Spec [4]；HELM [10]

层次	问法	不合格示例	合格示例
Mission / Product	为什么做这个模型？	“做一个更强模型”	“让开发者在真实 repo issue 上更高成功率完成修复，同时控制误改与执行副作用”
Capability	模型必须会什么？	“更会 Coding”	repo navigation / localization / patch / test / recovery
Behavior	它应该怎样做？	“更专业”	先读上下文；必要时用工具；不伪造执行结果；高风险操作前遵守权限边界
Constraint	哪些结果不可接受？	“更安全”	不越权修改范围外文件；不把未执行测试宣称通过；不输出不允许的内容

层次	问法	不合格示例	合格示例
Task Distribution	在哪些输入上达到目标？	“多做点训练数据”	70% repo tasks + 20% unit tasks + 10% adversarial/recovery；按难度分层采样
Reward/Eval	怎么证明达成？	“分数更高”	task success + patch correctness + regression rate + tool errors + latency + constraint violations

Know-Why 算法只会优化你给出的 signal。Objective 定义错误时，训练系统不会“理解真实意图并自动纠正”，而会把错误 proxy 做得越来越好。

2. 目标定义的六层模型：从愿景到可执行契约

一个成熟 Objective System 应当具备“可追踪性”：任何训练样本、reward 项和 eval 指标，都能向上追溯到某个产品目标或风险约束；任何高层目标，也能向下找到至少一个训练数据来源和至少一个独立验收指标。

层	定义	关键产物	验收问题
L1 Mission/Product Objective	模型创造什么真实价值、为谁创造	Objective Charter	如果模型变好，哪类用户结果会改变？
L2 Capability Spec	模型必须具备哪些技能	Capability Matrix	能力是否互斥/重叠？粒度是否可评测？
L3 Behavior Spec	模型在上下文中如何选择行为	Rules / Defaults / Examples	冲突时是否有优先级？边界是否可解释？
L4 Constraint & Risk Spec	不可突破的边界与风险预算	Constraint Matrix	哪些是 hard constraint，哪些是 soft preference？
L5 Task Taxonomy & Distribution	在哪些任务/难度/域上学习	Task Taxonomy + Mixture	训练分布是否代表目标分布并覆盖长尾？
L6 Reward / Eval Contract	如何测量、评分、上线	Graders + Metrics + Gate	训练 signal 与独立 eval 是否分离？

OpenAI Model Spec 提供了一个很有代表性的现代行为规范结构：先给出高层目标和通用原则，再通过不同 authority level 的指令来处理冲突，并进一步给出具体行为规则与默认值。[4] 这说明“目标”与“规则/默认行为”必须分层，因为高层目标太抽象，无法直接解决复杂冲突。

Constitutional AI 则从另一个方向说明：原则列表可以成为训练监督的上游。模型先依据 constitution 对自身输出进行 critique/revision，再将这些规范用于偏好与 RL 阶段。[3] 这使“行为规范”从文档变成了数据生成机制。

3. Capability Spec：把“会什么”拆成可训练的能力单元

Capability Spec 解决的是 competence，而不是风格。一个好能力定义必须满足四个条件：可观测、可分解、可泛化、可形成独立评测。不要用“智能”“推理强”“coding 好”这种无法直接变成数据 schema 的词作为最终能力节点。

能力层级	示例	可观测行为	典型 Eval
Domain	Software Engineering	能处理真实代码库问题	SWE-bench 类 repo issue
Capability	Repository Understanding	定位相关文件/函数/依赖	retrieval/localization accuracy
Sub-capability	Bug Localization	从 issue 与 test 失败定位根因	top-k file/function hit
Action Skill	Precise Editing	对正确位置产生最小 patch	patch apply + diff quality
Recovery Skill	Debug after Failure	测试失败后解释并修正	recovery success rate
Meta Skill	Stop / Escalate	知道何时缺信息、需澄清或停止	unnecessary action / false completion rate

FLAN 的结果提供了一个关于能力覆盖的早期证据：在多种任务上进行 instruction tuning 能提升未见任务的 zero-shot 泛化，并且任务数据集数量是关键因素之一。[9] 对目标定义的启发是：Capability Spec 不仅定义“主任务”，还要定义足够宽的邻近任务，使模型学到可迁移的行为模式，而不是记住单一 benchmark。

Know-How：能力节点的定义测试 对每个能力节点问三个问题：(1) 能否构造至少 100 个难度不同的任务？(2) 能否有两个相互独立的 grader？(3) 失败后能否明确归因到这个节点？任一答案为“否”，粒度通常仍然不合适。

4. Behavior Spec：把“应该怎么做”写成可训练行为规范

Capability 决定模型能不能完成任务，Behavior 决定模型在有多种可能行为时选择哪一种。例如一个 Coding Agent 既可能“立即修改文件”，也可能“先搜索调用关系”；两种行为都可能完成任务，但后者在复杂 repo 中通常更稳健。Behavior Spec 就是在定义这种 policy prior。

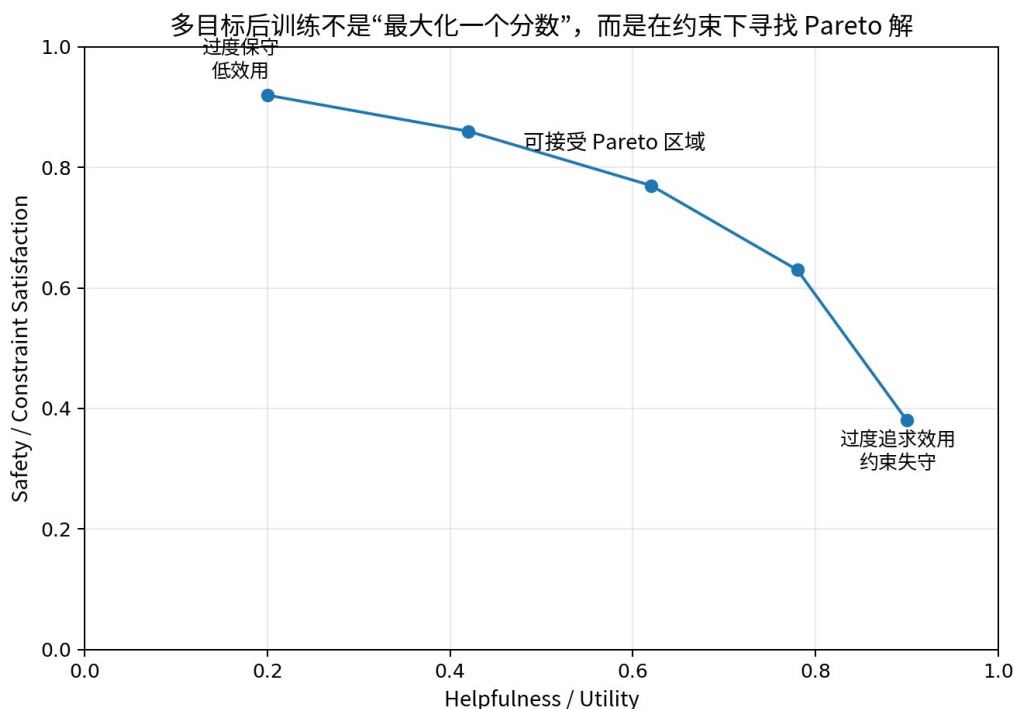
InstructGPT 的 labeler 指令非常典型：helpful 被具体化为遵循用户意图、清晰回答、在输入含糊时澄清、避免冗长；truthful 包括不凭空添加输入中没有的信息、不制造明显虚假事实；harmless 则进一步定义对现实伤害和不良建议的约束。[1] 更关键的是，论文还记录了 labeler 指令在项目中发生变化，训练阶段与最终评测阶段对 helpfulness、truthfulness、harmlessness 的优先级并不完全相同。[1] 这说明 Behavior Spec 必须版本化。

Behavior 组件	定义方法	工程例子
Intent Interpretation	如何理解显式/隐式意图	输入模糊时澄清；错误前提时纠正而不是顺着错误前提
Action Policy	先做什么、后做什么	先 read/search，再 edit；修改后必须 test
Communication	如何呈现结果	区分已验证事实、推断和未验证状态
Uncertainty	什么时候承认不知道	证据不足时不伪造；给出需要的额外信息
Autonomy Boundary	多大范围可自主执行	只在授权范围内修改；对高影响动作设置更严格 gate
Error Recovery	失败后怎么做	读取错误、重新定位、有限次数重试，避免循环
Stop Condition	什么时候结束	达到验收条件才宣告完成；无法满足时明确残余风险

OpenAI Model Spec 的启发在于“冲突解决机制”本身也是 Behavior Spec 的一部分：高层目标并不能直接处理多个指令之间的冲突，因此需要显式 authority/priority。对企业模型同样适用：系统约束、开发者要求、最终用户请求、第三方工具输出之间应有明确定义的优先级。[4]

5. 多目标、约束与冲突：Objective 不是单一 scalar

真实后训练几乎总是 multi-objective。常见目标包括 correctness、helpfulness、truthfulness、safety、style、latency、token cost、tool efficiency、format adherence、autonomy 等。把它们粗暴压成一个 scalar 往往会掩盖不可接受的 trade-off。



资料依据：概念图；多目标冲突依据 Anthropic HH-RLHF [2] 与 OpenAI safe-completion [7]

Anthropic 的 HH-RLHF 实验显示 helpfulness 与 harmlessness 存在明显张力：只训练其中一个目标会显著损害另一个；研究还观察到 over-optimizing harmlessness 导致模型对许多敏感但并不需要强拒绝的输入产生夸张保守响应。[2] 这不是“模型不够聪明”，而是目标组合与数据分布定义的问题。

OpenAI 的 safe-completion 更明确地把目标写成 constrained optimization：在安全约束内最大化 helpfulness。官方说明将训练参数拆为 safety constraint 与 helpfulness maximization，而不是只做二元 comply/refuse 分类。[7] 这是一种非常值得迁移到一般后训练的 Objective 模式。

目标类型	建议表达	是否允许补偿
Hard Constraint	违反即 fail / gate 不通过	不允许被其他高分抵消
Soft Constraint	超过阈值扣分或降级	有限允许
Primary Objective	在满足约束后主要最大化	允许与次目标权衡
Secondary Objective	体验/成本/风格等优化	允许
Diagnostic Metric	只用于归因，不直接训练	不参与总奖励

```
# 推荐：不要只写一个总分
objective:
  hard_constraints:
    - no_scope_violation
    - no_false_completion
  primary:
    - task_success
  secondary:
    - minimal_patch
    - tool_efficiency
```



```
- response_clarity
diagnostics:
- localization_accuracy
- retry_count
```

6. Objective -> Data：目标如何决定训练数据分布

后训练的数据分布不是“越多越好”，而是 Objective 的采样实现。InstructGPT 直接从 API 使用分布中构造 prompt dataset，并报告真实 use-case 类别；论文还发现公共 NLP 数据集并不能充分代表真实 API 使用分布。[1] 这说明目标定义必须明确“目标输入分布”，否则 benchmark 上的能力并不自动等价于产品目标。

Objective 维度	Data 设计问题	常见错误
用户/产品分布	谁在用？真实 prompt 占比多少？	只用公开 benchmark
任务类型	任务 taxonomy 是否覆盖主流程？	只训最终 answer，不训中间 action
难度	easy/medium/hard 如何分配？	大量 easy data 淹没 hard signal
长尾	少见但高影响 failure 是否过采样？	完全按自然频率，导致关键长尾学不到
负例	什么错误行为需要显式 rejected？	只有正例，模型不知道边界
恢复	是否包含 tool fail / test fail / incomplete context？	训练世界过于理想化
版本	数据是否与当前 behavior spec 同版本？	旧 policy data 持续污染新目标

Objective-driven sampling 最实用的做法不是先拼数据再算比例，而是先为 Capability/Behavior 节点分配目标权重，再把每个节点映射到数据源、难度、正负例与采样率。数据 mixture 就是 Objective 的第一种“编译结果”。

7. Objective -> Reward：把规范编译成可优化信号

Reward Contract 的任务是把“什么叫好”转化成机器可计算或可稳定标注的信号。这里最危险的误区是：只要某个 grader 与人类偏好相关，就可以无限优化。事实上 reward 是 proxy；一旦优化压力过大，模型会寻找 grader 的捷径。

OpenAI 的 Rule-Based Rewards 是“spec -> reward”工程化的代表：先定义关于输出行为的 propositions，再用规则组合成 ideal / less_good / unacceptable 等评价，最后由 grader 对规则满足情况

评分，并将该信号与 helpfulness reward 一起用于 RL。[5] 这说明 Reward 设计之前必须先有可解释的 Behavior Spec。

Deliberative Alignment 又向前一步：安全 specification 不只用于生成标签，而是直接用于合成训练数据与 policy-aware reward model。SFT 让模型学习 specification 内容及如何推理，RL 再通过可访问 specification 的 reward model 强化这种行为。[6] 这实际上把“目标文档”提升为训练时的一等对象。

Reward 类型	最适合的目标	优点	风险
Exact / Verifier	数学答案、单测、schema	客观、稳定	只覆盖可验证部分
Rule-based	格式、边界、明确 policy	可解释、易更新	复杂主观质量难覆盖
Human Preference	整体体验、写作质量	贴近人类判断	昂贵、噪声、群体代表性
Model Judge	大规模主观评分	扩展快	judge bias / self-preference
Composite Reward	多目标	可统一优化	权重隐藏 trade-off / reward hacking

DeepSeek-R1 从 capability 目标角度提供另一个重要启发：对 reasoning 任务，目标可优先采用可验证的最终正确性，而不是强行指定“正确思维过程”的每一步；R1-Zero 的公开结果同时表明，仅靠能力 reward 可能出现可读性、语言混杂等行为问题，因此后续 R1 加入 cold-start 与多阶段训练修正这些非能力目标。[8] 这恰好说明 Capability Objective 与 Behavior Objective 必须拆开。

8. Objective -> Eval：评测不是训练后的报告，而是目标定义的一部分

一个目标如果没有独立 Eval，就还不是工程目标。最关键的原则是：training signal 与 acceptance eval 应尽量解耦。若 reward grader 与上线 eval 完全相同，优化会快速过拟合该 grader，而真实能力不可知。

HELM 提倡 scenario taxonomy 与 multi-metric evaluation：在不同使用场景上同时测 accuracy、robustness、fairness、toxicity、efficiency 等多种 desiderata，以暴露 trade-off。[10] 对目标定义的核心启发是：Eval Spec 必须是“场景 × 指标”的矩阵，而不是单一总分。

OpenAI 的 Evals 实践强调用评测发现模型短板、避免回归，并允许多种 grader 类型；对于 Agent 类系统，还需要 trace-level 或 environment-level 的任务成功评测，而不仅是最终文本相似度。[12]

Eval 层	目的	示例指标	是否作为上线 Gate
Outcome Eval	验证最终任务结果	task success / pass rate	是
Constraint Eval	验证不可接受行为	violation rate / false completion	是

Eval 层	目的	示例指标	是否作为上线 Gate
Capability Eval	定位具体能力	localization / planning / tool args	部分
Behavior Eval	验证交互策略	clarification / recovery / stop	部分
Robustness Eval	分布外与对抗性	OOD / perturbation / long-tail	是
Efficiency Eval	成本与时延	tokens / tool calls / latency	视产品
Regression Eval	保护已有能力	baseline suite delta	是

Eval Contract 每一个上线目标都应写明：metric、dataset/version、grader/version、sampling、置信区间、最低阈值、最大回归、hard failure 定义。否则“分数 80”没有工程意义。

9. 深入案例：把“提高 Coding Agent 能力”完整拆成 Objective Tree

下面给出一个可以直接用于 Post-Training 项目的分解方式。SWE-bench 的价值就在于把“代码生成”提升为真实 GitHub issue resolution：模型需要理解 codebase、跨文件推理、编辑代码并在执行环境中验证。[11] 但单一 SWE-bench 成功率仍不足以定义完整 Coding Agent 目标，因此还需要行为、约束和效率指标。

一级能力	二级能力	可训练行为	主要 Eval
Repository Understanding	Repo Navigation	找到相关目录、文件、symbol	file/symbol localization
Repository Understanding	Dependency Reasoning	理解调用与依赖关系	dependency QA / trace
Problem Solving	Issue Interpretation	把自然语言 issue 转成可验证要求	requirement coverage
Problem Solving	Root Cause Analysis	定位 bug 原因而非只改症状	root-cause rubric
Editing	Patch Generation	生成最小且正确 patch	apply + tests + diff size
Testing	Test Selection	选择相关测试	test relevance
Testing	Execution	运行并正确解释测试	execution truthfulness
Recovery	Debug Loop	失败后根据 observation 修正	recovery success

一级能力	二级能力	可训练行为	主要 Eval
Tool Use	Selection & Args	正确 read/grep/edit/shell	tool accuracy
Long Horizon	Plan & State	多步保持目标与状态	task success / step efficiency
Stop	Completion Judgment	真正满足验收再结束	false completion rate

目标类别	Coding Agent Objective 示例
Mission	在真实 repo issue 上提高端到端解决率，同时控制错误修改、虚假完成和执行成本
Primary Objective	Verified task success
Hard Constraints	不得越出允许 workspace；不得把未执行/失败测试报告为通过；不得破坏无关行为
Behavior Objectives	先定位再编辑；编辑后验证；失败后基于 observation 修正；必要时澄清
Efficiency	tool calls / generated tokens / wall-clock 在预算内
Robustness	不同 repo、语言、issue 风格、长上下文、工具失败条件下维持性能
Regression Budget	已有代码生成/QA 能力回归不超过预设阈值

```
capability_spec:
  domain: coding_agent
  capabilities:
    repository_understanding:
      weight: 0.18
      evals: [file_localization, symbol_localization]
    bug_fixing:
      weight: 0.32
      evals: [verified_issue_resolution]
    tool_use:
      weight: 0.15
      evals: [tool_selection, argument_validity]
    testing_recovery:
      weight: 0.20
      evals: [test_interpretation, recovery_success]
    completion_judgment:
      weight: 0.15
      evals: [false_completion, unnecessary_steps]

  hard_constraints:
    - workspace_scope_violation == 0
    - false_test_claim_rate <= 0.1%
```

10. 不同模型类型，目标定义方式也不同

模型类型	Primary Objective	Behavior Objective	核心约束	最关键 Eval
通用 Chat Assistant	用户任务完成/偏好	遵循意图、真实、清晰、适度澄清	安全、隐私、不可虚假	真实用户分布偏好 + truthfulness
Reasoning Model	可验证问题正确率	有效搜索、自检、在预算内停止	不靠格式/长度作弊	accuracy@budget / pass@k / verifier
Coding Agent	端到端 repo task success	read-search-edit-test-recover	workspace/side-effect/false completion	environment task success
Tool Agent	正确选择和组合工具	观察结果后更新计划	权限、不可逆动作边界	tool accuracy + task success
Enterprise Domain Model	业务流程成功率	遵守企业 SOP/格式	合规、数据边界	workflow success + policy adherence

Reasoning 模型尤其要防止把“更长的 CoT”误当成目标。真正 Objective 应写成“在固定预算下提高可验证正确率”或“在相同正确率下减少 reasoning cost”。长度只是诊断变量，不应默认成为 reward。

通用 Chat Assistant 则不同：许多任务没有唯一 ground truth，因此 preference、rule-based grading 和真实用户分布更重要；同时必须处理多个价值目标的优先级。InstructGPT 与后续 Model Spec 都体现了这种“behavioral objective”主导的特点。[1][4]

11. 目标定义的 12 类典型失败模式

失败模式	症状	根因	修复
1. Vague Objective	“更强/更智能”	不能转成数据与 Eval	拆 Capability + Behavior + Metric
2. Benchmark = Objective	榜单涨、产品不涨	目标分布不同	真实任务分布 + benchmark 仅作 proxy
3. Single Scalar	总分高但关键行为退化	trade-off 被平均	hard constraint + Pareto metrics
4. Reward/Eval Same Grader	reward 涨、独立人评不涨	grader overfit	独立 holdout grader
5. Behavior Unspecified	能力强但交互不稳定	只定义“会什么”	单独 Behavior Spec
6. Priority Missing	不同目标互相打架	无冲突规则	authority / priority / constraint
7. Data-Objective Mismatch	训练很多无关 easy case	mixture 不受目标约束	objective-driven sampling
8. Long-tail Missing	平均很好，关键场景失败	按自然频率训练	risk-weighted oversampling

失败模式	症状	根因	修复
9. Metric Without Threshold	知道分数不知道能否上线	无 acceptance contract	明确 pass/fail gate
10. No Regression Budget	新能力换来旧能力损失	只看新增指标	建立 baseline regression suite
11. Spec Drift	不同团队使用不同“好”的定义	版本不一致	spec versioning + data lineage
12. No Failure Taxonomy	只知道失败率	无法生成下一轮数据	failure -> capability node -> data

最危险的反模式 “先训一个版本看看，再决定我们要什么。” 这会把实验结果变成目标定义的替代品。正确流程是：先写出最低可执行 Objective Contract，再允许训练结果反过来修订 Objective，但所有变更必须版本化。

12. 工程落地：Objective Contract 模板

12.1 Objective Charter（一页纸）

字段	建议内容
Model / Version	模型族与目标版本
Target Users	开发者、终端用户、内部 Agent 等
Top 3 Jobs-to-be-Done	模型最核心的三个真实任务
Primary Outcome	一个最重要的端到端结果指标
Hard Constraints	绝不能被其他指标补偿的边界
Secondary Objectives	体验、成本、风格、效率
Target Distribution	域/任务/难度/语言/上下文长度
Known Trade-offs	明确哪些目标可能冲突
Eval Gate	上线必须达到的最低标准
Regression Budget	允许哪些已有能力最多退化多少
Owner / Review	产品、训练、评测、风险负责人

12.2 capability_spec.yaml

```
version: objective-v1.3
model_family: agent-model
mission:
  target_user: software_engineer
  job: resolve_repository_issues

capabilities:
- id: repo_localization
  priority: P0
  target_distribution: [python, typescript]
  metrics:
    file_hit_at_5: ">= 0.92"
    symbol_hit_at_10: ">= 0.88"
- id: verified_patch
  priority: P0
  metrics:
    task_success: ">= baseline + 8pp"

constraints:
- id: no_false_completion
  metric: false_completion_rate
  threshold: "<= 0.2%"
  hard_gate: true
```

12.3 behavior_spec.yaml

```
behavior_spec:
  principles:
    - understand_before_act
    - verify_before_claim
    - minimize_unnecessary_side_effects
  defaults:
    before_edit:
      - inspect_relevant_context
    after_edit:
      - run_relevant_validation
    on_failure:
      - inspect_observation
      - revise_hypothesis
      - retry_with_budget
    completion:
      - only_claim_success_if_acceptance_evidence_exists
  conflict_priority:
    - hard_constraints
    - task_correctness
    - user_instruction
    - efficiency
    - style
```

12.4 eval_spec.yaml

```
eval_spec:
  primary:
    - name: verified_task_success
      dataset: repo_eval_v7
```

```
grader: environment_verifier_v3
threshold: 0.48
confidence: 95%
hard_gates:
- name: scope_violation_rate
  threshold: 0.0
- name: false_completion_rate
  threshold: 0.002
regressions:
- name: general_instruction_following
  max_drop_pp: 1.0
diagnostics:
- tool_selection_accuracy
- localization_hit_rate
- recovery_success
- median_tool_calls
```

12.5 Objective Review 流程

1. 产品负责人写 Mission / Target User / Jobs-to-be-Done，不直接写训练算法。
2. 研究/训练团队将 Mission 拆成 Capability 与 Behavior 节点，并标注 P0/P1/P2。
3. Eval 团队为每个 P0 节点定义独立测量方式；如果测不了，目标继续拆。
4. Data 团队为每个节点定义数据源、难度、正负例与采样比例。
5. RL/Alignment 团队为可优化目标定义 reward；hard constraint 与 soft objective 分离。
6. 在训练前做一次“spec consistency review”：Data/Reward/Eval 是否指向同一个 Objective。
7. 训练后先看 hard gate，再看 primary outcome，再看 regression，最后看诊断指标。
8. 失败样本必须归入 Capability/Behavior taxonomy，成为下一轮数据而不是只进入报告。

13. Objective System 成熟度模型

Level	团队状态	典型特征	下一步
L0 KPI-driven	“提升某 benchmark”	无行为规范、无数据 lineage	建立 Objective Charter
L1 Capability-driven	已有能力矩阵	能力可测，但行为/约束弱	加入 Behavior + hard constraints
L2 Spec-driven	Capability/Behavior/Eval 成体系	目标已可版本化	Data/Reward 自动映射
L3 Contract-driven	Data/Reward/Eval 同源	上线 gate、回归预算完整	失败自动归因
L4 Flywheel-driven	目标 -> 训练 -> 失败 -> 数据闭环	持续线上反馈与版本治理	动态任务分布与 risk-aware allocation

成熟度提升的核心，不是增加更多模型算法，而是提高“目标到信号”的可追踪性。最终理想状态是：某个线上失败能够追溯到 Objective 节点；节点能对应 Eval；Eval 能对应数据与 reward；下一轮训练后能明确验证该节点是否改善且没有跨节点回归。

14. 目标定义最终检查清单

检查项	Pass 条件
目标是否指向真实用户结果?	不是“模型分数更高”，而是明确用户/任务结果
是否区分 Capability 与 Behavior?	“会什么”和“怎么做”有不同节点
是否声明 hard constraints?	关键边界不能被总分补偿
冲突是否有优先级?	多目标冲突时有明确规则
目标输入分布是否明确?	任务、域、难度、语言、上下文长度有定义
每个 P0 目标是否可评测?	至少一个独立 eval，最好有第二种交叉验证
Reward 是否只是 proxy?	明确 reward 与真实目标的差距和防 overoptimization 方法
训练与验收是否解耦?	独立 holdout / human / environment eval
是否有回归预算?	新增能力不能无限交换已有能力
是否版本化?	objective/data/reward/eval 都有 version 与 owner
失败是否可归因?	失败能映射到 taxonomy 节点
是否支持迭代?	failure -> data -> training -> eval 闭环明确

最终判断 如果团队无法回答“这个训练样本属于哪个 Objective 节点”“这个 reward 对应哪个行为要求”“这个 Eval 失败后该生成什么数据”，说明目标定义仍然停留在口号，而没有成为后训练系统。

参考资料与研究依据

来源	本手册使用方式
[1] Ouyang et al., 2022	Training language models to follow instructions with human feedback. arXiv:2203.02155. 关键依据：预训练目标与用户意图目标不一致；helpful/honest/harmless 定义；真实 API prompt 分布；labeler instruction 与 trade-off。
[2] Bai et al., 2022	Training a Helpful and Harmless Assistant with Reinforcement Learning from Human Feedback. arXiv:2204.05862. 关键依据：helpfulness/harmlessness 多目标张力、数据 mixture 与 loss weighting、过度优化 harmlessness 的行为退化。
[3] Bai et al., 2022	Constitutional AI: Harmlessness from AI Feedback. arXiv:2212.08073. 关键依据：原则/constitution 作为监督源，通过 critique/revision 与 AI preference 进入后训练。
[4] OpenAI, 2025-10-27	Model Spec. 关键依据：从高层目标到规则/默认行为、instruction authority、冲突优先级、模型行为规范的版本化与治理。
[5] OpenAI, 2024-07-24	Improving Model Safety Behavior with Rule-Based Rewards. 关键依据：propositions -> rules -> grader -> reward；显式规则将行为规范编译成 RL signal。
[6] OpenAI, 2024-12-20	Deliberative alignment: reasoning enables safer language models. 关键依据：直接教授 specification；spec 驱动合成 SFT 数据和 policy-aware reward。
[7] OpenAI, 2025-08-07	From hard refusals to safe-completions: toward output-centric safety training. 关键依据：在 safety constraint 内最大化 helpfulness；把多目标写成 constrained optimization。
[8] DeepSeek-AI, 2025	DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. arXiv:2501.12948. 关键依据：能力 reward 与行为质量分离；纯 RL 的 reasoning 能力和可读性/语言混杂问题；multi-stage 修正。
[9] Wei et al., 2021/2022	Finetuned Language Models Are Zero-Shot Learners (FLAN). arXiv:2109.01652. 关键依据：多任务 instruction tuning 与未见任务泛化；任务数量的重要性。
[10] Liang et al., 2022	Holistic Evaluation of Language Models (HELM). arXiv:2211.09110. 关键依据：scenario taxonomy、multi-metric evaluation、暴露 trade-off。
[11] Jimenez et al., 2023	SWE-bench: Can Language Models Resolve Real-World GitHub Issues? arXiv:2310.06770. 关键依据：从 code generation 到 repo-level issue resolution 的能力目标升级。

来源	本手册使用方式
[12] OpenAI	Evals / Graders documentation and GPT-4 Evals discussion. 关键依据：eval 用于发现短板、避免回归；多种 grader 类型与可复用评测结构。

说明：本手册中的 Objective Contract、六层模型、模板与成熟度模型，是基于上述公开资料进行的工程化综合与推导，不代表任一单一机构的官方框架。文中引用的案例只用于解释目标定义方法，不复现任何不必要的敏感操作细节。

参考链接

InstructGPT: <https://arxiv.org/abs/2203.02155>

Helpful & Harmless RLHF: <https://arxiv.org/abs/2204.05862>

Constitutional AI: <https://arxiv.org/abs/2212.08073>

OpenAI Model Spec: <https://model-spec.openai.com/2025-10-27.html>

OpenAI Rule-Based Rewards: <https://openai.com/index/improving-model-safety-behavior-with-rule-based-rewards/>

OpenAI Deliberative Alignment: <https://openai.com/index/deliberative-alignment/>

OpenAI Safe Completions: <https://openai.com/index/gpt-5-safe-completions/>

DeepSeek-R1: <https://arxiv.org/abs/2501.12948>

FLAN: <https://arxiv.org/abs/2109.01652>

HELM: <https://arxiv.org/abs/2211.09110>

SWE-bench: <https://arxiv.org/abs/2310.06770>